
LIGHTWEIGHT PLUGGABLE AI ENGINE

EduFish 设计与开发文档

智慧教育分析引擎 — 为传统教育平台注入多智能体分析与动态推演能力

EduFish 是一款轻量化、可插拔的 AI 赋能引擎。宿主平台接入后，瞬间激活多智能体协同、碎片化信号串联、证据图谱生成与干预策略推演的完整能力。文档面向评审场景，说明引擎架构、智能体协作、数据治理和部署方案。

摘要

EDUFISH 智慧教育分析引擎是一款专为现代教育生态打造的轻量化、可插拔 AI 赋能中枢。针对传统教学平台数据分析滞后、干预手段单一的痛点，EDUFISH 提出了全新的解决方案。它并非一个封闭的系统，而是被设计为能够极速接入并集成至各类现有教育平台的基础设施。

接入本引擎后，宿主平台将瞬间激活强大的多智能体（Agent）协同与动态预测分析能力。EDUFISH 能够实时捕获并串联出勤、成绩、课堂交互等碎片化信号，自动推演生成可视化的“教学质量证据图谱”。它不仅能精准捕捉教学风险，还能直接为教师提供干预策略的推演结果——例如直观展示“增加实验复盘”后学习达成率的提升预测。EDUFISH 致力于通过极低成本接入方案，让每一个传统教育系统都能迅速拥有智能化的“大脑”。

核心价值：EDUFISH 不是另一个教育平台，而是一个引擎。它不替代宿主系统，而是为其注入多智能体分析、证据图谱构建、干预策略推演和结构化报告生成能力。接入一次，宿主平台即刻拥有从数据到决策的智能闭环。

评审关注点

评审维度	说明
引擎定位	轻量化、可插拔的 AI 赋能中枢，非封闭平台，可极速接入现有教育系统
技术实现	多智能体协同、动态预测分析、证据图谱生成、API-First 架构
接入成本	极低成本接入方案，宿主平台无需重构即可获得完整智能分析能力
可信机制	关键结论保留证据来源、置信说明、风险提示和人工复核入口
落地路径	引擎核心先行，逐步扩展接入层、嵌入组件和多平台适配

表 1：EDUFISH 面向评审的核心说明维度。

一、产品定位

1.1 问题定义

传统教育平台已经积累了大量数据，但这些数据通常分散在教务系统、问卷系统、作业平台、课堂互动工具和人工整理的表格中。这些平台的共同痛点是：数据分析严重滞后于教学过程，干预手段停留在事后总结，缺少实时的、可推演的、可验证的智能分析能力。

管理者真正需要的不是更多图表，而是能够回答三个问题的系统：课程为什么好或不好，教师和学生分别需要什么支持，下一步干预是否有证据。然而，为每一个教育平台从零构建这样的能力，成本高、周期长、技术门槛高。

EDUFISH 的价值在于：它是一个独立部署的引擎，宿主平台通过极低成本的接入，即可瞬间获得多智能体协同分析、碎片化信号串联、证据图谱生成和干预策略推演的完整能力。数据进入引擎后先被归一化，再由教育专用智能体提取主题、趋势、风险和建议，最后生成面向不同角色的报告与可执行任务。

1.2 目标用户

角色	核心诉求	引擎应提供的结果
宿主平台方	以极低成本获得智能分析能力，无需重构现有系统	API 接入、SDK 集成、嵌入式组件、Webhook 事件驱动
院系管理者	快速识别课程质量、师生反馈和教学风险	课程画像、风险排序、治理建议、学期报告
课程负责人	理解课程内容、节奏、作业与评价是否匹配	课程健康度、反馈主题、改进任务、证据引用
一线教师	知道学生哪里卡住、哪些教学动作有效，干预后效果如何	教学反馈摘要、学生困惑聚类、干预策略推演、可执行微调建议
教务与质保团队	把分散数据转化为可审计材料	标准化模板、数据质量检查、报告归档、权限与审计

表 2：EDUFISH 的角色和交付物。

1.3 产品边界

EDUFISH 的定位不是通用报表工具，也不是另一个教育管理平台。它是一个可插拔的 AI 引擎，专为宿主平台提供智能分析的底层能力。

EDUFISH 不替代宿主平台的任何功能，而是在宿主平台授权数据的基础上完成整理、分析、解释与报告生成。宿主平台保留全部用户关系、业务流程和数据主权。

EDUFISH 不以模型输出替代专家判断；关键结论需要提供证据来源、置信说明和人工复核入口。

EDUFISH 的交互界面应服务于证据阅读和决策支持，避免复杂表单、过度装饰和与评估任务无关的信息干扰。

二、引擎架构与接入模型

2.1 引擎分层

EDUFISH 采用四层架构：接入层负责与宿主平台对接，数据层完成多源数据归一化与证据索引，智能层驱动多智能体协同分析与推演，输出层生成角色化报告与可视化图谱。各层保持清晰的输入输出关系，使宿主平台的接入、引擎的分析和用户的决策形成可解释的技术闭环。

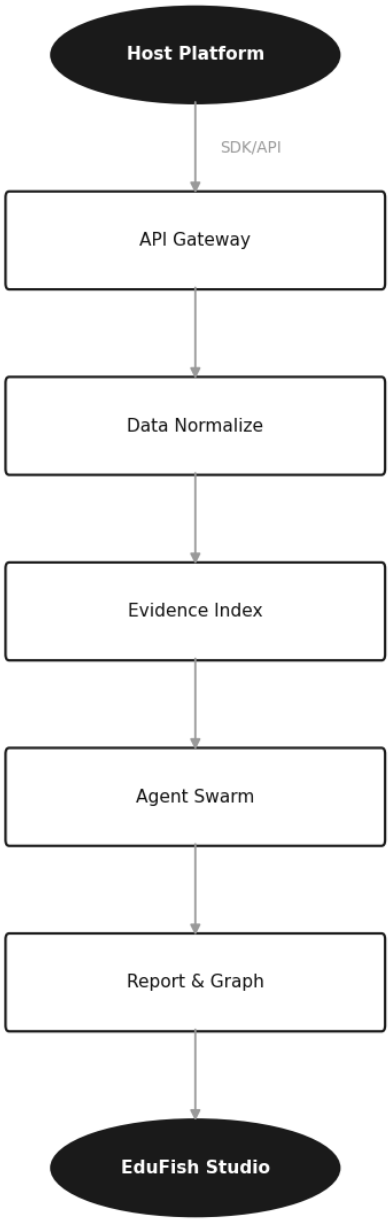


Figure 1: EDUFISH 引擎端到端处理流程：从宿主平台数据接入到分析报告和应用呈现。

2.2 分层职责

层级	职责	设计要点
----	----	------

接入层	接收宿主平台通过 API、SDK 或 Webhook 推送的多源数据	API-First 设计，支持多来源输入，保留数据来源与时间范围
数据层	统一课程、教师、学生、时间和评价字段，建立可追溯证据索引	降低不同系统字段差异对分析结果的影响，每项结论均可回溯
智能层	驱动反馈分析、课程质量、教学评价、风险治理等专用智能体协同工作	通过专用智能体分工提升分析的稳定性和专业性，支持动态预测推演
输出层	按评审、管理、教师等场景生成结构化报告与可视化证据图谱	输出摘要、证据、风险、建议、干预策略推演和后续行动

表 3：EDUFISH 引擎分层设计。

2.3 接入方式

EDUFISH 提供多种接入方式，宿主平台可根据自身技术栈和集成深度选择最合适的方案。所有接入方式共享同一套引擎核心能力，区别仅在于数据推送和结果获取的交互模式。

接入方式	适用场景	集成深度
RESTful API	任何支持 HTTP 的平台，适合快速验证和轻量集成	标准：API 版本化管理（/api/v1/），API Key 认证，速率限制保护，结构化错误响应
SDK（Python / JS）	宿主平台使用 Python 或 Node.js 技术栈，需要深度集成	已交付：封装 API 调用，类型安全，错误处理，pip/npm 安装即用
Webhook 事件驱动	宿主平台需要实时或准实时的异步分析通知	已交付：HMAC-SHA256 签名，自动重试，事件类型包括 analysis.completed / analysis.failed / report.ready
嵌入式前端组件	宿主平台需要直接展示分析结果和证据图谱	已交付：Vite library mode 构建，ES/UMD 双格式，CSS 变量隔离，自适应画布，Web Worker 内联

表 4：EDUFISH 接入方式对比。

2.4 接入流程

宿主平台接入 EDUFISH 的标准流程分为四步：注册引擎实例并获取凭证、配置数据映射规则、推送数据触发首次分析、嵌入结果展示组件。整个过程可在数小时内完成，无需修改宿主平台的数据库结构或业务逻辑。

JSON

```
{
  "integration_steps": [
    {
      "step": 1,
      "action": "Register engine instance",
      "output": "API_KEY + ENGINE_ENDPOINT"
    },
    {
      "step": 2,
      "action": "Configure data mapping",
      "output": "field_mapping.json"
    },
    {
      "step": 3,
      "action": "Push data and trigger analysis",
      "output": "analysis_id + evidence_graph"
    },
    {
      "step": 4,
      "action": "Embed result components",
      "output": "<EduFishGraph> <EduFishReport>"
    }
  ]
}
```


2.5 企业级引擎特性

EDUFISH 引擎在核心分析能力之上，提供了一整套企业级基础设施，使宿主平台能够在生产环境中安全、可靠地运行引擎服务。这些特性是引擎“开箱即用”承诺的技术保障。

API 版本管理

所有引擎端点统一使用 `/api/v1/` 版本化前缀。引擎通过 `X-Engine-Version` 和 `X-Engine-Name` 响应头标识自身版本和实例名称。原有 `/api/` 前缀路径返回 410 Gone 状态码并引导调用方迁移，确保平滑升级。

认证与安全

引擎通过 `X-API-Key` 请求头进行 API Key 认证，支持按路径配置鉴权范围。关键端点（数据分析、报告生成）默认要求认证，健康检查端点豁免。认证失败时返回统一的 401 错误响应。

速率限制

基于 Flask-Limiter 的 in-memory 速率限制，无需外部 Redis 依赖即可运行。全局默认限制为 60 次/分钟，分析端点（`/edu/analysis/*`）限制为 10 次/分钟，报告导出限制为 5 次/分钟。超限时返回 429 Too Many Requests 及标准错误载荷。速率限制可通过环境变量 `RATE_LIMIT_ENABLED` 灵活开关。

Webhook 事件驱动

异步分析任务完成后，引擎主动向宿主平台配置的 `webhook_url` 发送事件通知。事件类型包括 `analysis.completed`、`analysis.failed`、`report.ready`。每个请求携带 HMAC-SHA256 签名（`X-Edufish-Signature` 头），宿主平台可验证来源可信性。失败时自动重试一次（5 秒延迟），结果写入 Job 错误日志。

多租户隔离

引擎支持通过 `X-Tenant-ID` 请求头进行租户级数据隔离。EduDataset、EduAnalysis、EduReport 三个核心模型内置 `tenant_id` 字段，所有数据库查询自动过滤当前租户。未提供租户标识的请求默认归于 “default” 租户。多租户功能可通过 `MULTI_TENANT_ENABLED` 环境变量按需开启，现有数据自动回填。

容器化部署

引擎后端提供 Dockerfile（Python 3.11-slim 基础镜像），配套 `docker-compose.yml` 编排文件实现一键启动。健康检查端点 `/health` 可用于容器编排平台的就绪探测。部署脚本 `setup.sh` 自动完成依赖安装、数据目录初始化和服务启动。

前端组件库化

知识图谱组件支持以 npm 包形式独立发布（@edufish/vue）。基于 Vite library mode 构建，输出 ES Module 和 UMD 双格式，CSS 变量带 fallback 确保在任意宿主样式中正确渲染。

染。Vue 3、D3、GSAP 作为 peer dependencies 外部化，不重复打包。Web Worker 完整内联（含 d3-force），无需额外文件部署。组件支持 ResizeObserver 自适应容器尺寸，SVG 和 Canvas 双渲染器自动切换。

JSON

```
{
  "enterprise_features": {
    "api_versioning": { "prefix": "/api/v1/", "legacy": "410 Gone", "headers": ["X-Engine-Version",
    "auth": { "method": "X-API-Key", "errors": "401 Unauthorized" },
    "rate_limiting": { "engine": "Flask-Limiter", "storage": "in-memory", "default": "60/min", "an
    "webhook": { "signing": "HMAC-SHA256", "retry": "1x @ 5s", "events": ["analysis.completed", "an
    "multi_tenancy": { "header": "X-Tenant-ID", "models": ["EduDataset", "EduAnalysis", "EduReport"
    "containerization": { "dockerfile": true, "compose": true, "healthcheck": "/health", "base_imag
    "component_library": { "package": "@edufish/vue", "formats": ["es", "umd"], "externals": ["vue"]
  }
}
```

企业级特性总结：API 版本化管理、认证鉴权、速率保护、事件驱动回调、多租户数据隔离、容器化部署、前端组件独立发布 — 全部就绪，引擎具备生产环境多租户部署能力。

三、设计原则

3.1 从表单转向智能工作台

教师工作界面和 AI 助教界面不应该呈现为一组输入框的堆叠。教育分析的情境更接近研究工作台：一边是材料和证据，一边是问题、假设、推理和结论。页面结构应减少封闭矩形，利用开放式区块、细线、错落模块和留白建立阅读节奏。

3.2 以证据为中心

任何课程评价、教师反馈或风险判断都必须能追溯到原始数据片段、聚合指标或课程知识证据。界面中不应只展示一个分数，而应展示分数背后的证据构成、时间范围、样本规模和异常来源。

3.3 低噪声动效

AI Pulse、Graph Overview 和神经网络可视化应使用克制微交互：缓慢漂移、轻量脉冲、悬停高亮、局部连线，而不是大面积闪烁或无意义的线条。动画的职责是提示系统正在推理、连接证据和形成图谱，不是装饰页面。

设计准则：少边框、少卡片、少填充色；多留白、多层级、多证据。图形动效只服务于状态和关系，不制造视觉噪声。

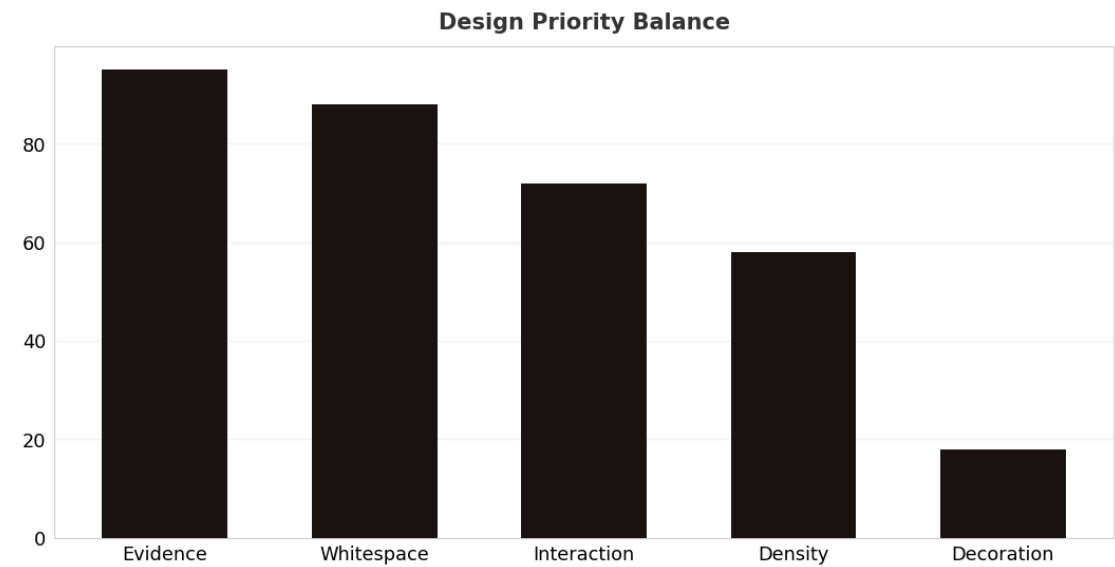


Figure 2: 设计优先级示意：证据、留白和结构优先于装饰性视觉元素。

四、多智能体协同分析

4.1 智能体分工

教学质量评估涉及反馈理解、课程结构分析、风险识别、证据组织和报告撰写等不同类型任务。EDUFISH 采用多智能体协同方式，将复杂分析拆解为职责明确的子任务，使每一类判断都有对应的数据输入、分析产出和证据依据。宿主平台接入引擎后，这些智能体自动激活，无需宿主平台额外配置。

智能体	处理对象	分析产出
反馈分析智能体	学生评教、开放反馈、课堂互动文本	主题聚类、情绪趋势、典型证据、噪声提示
课程质量智能体	课程大纲、成绩、作业、考勤、章节进度	课程健康度、难度节奏、内容覆盖、风险点
教学评价智能体	教师授课数据、反馈、课堂参与、历史趋势	教学优势、待改进项、支持建议
风险治理智能体	异常分布、低参与度、争议反馈、数据缺口	风险等级、触发原因、复核建议
报告生成智能体	上游分析结果与证据索引	角色化报告、摘要、行动计划、附录

表 5：EDUFISH 的教育专用智能体拆分。

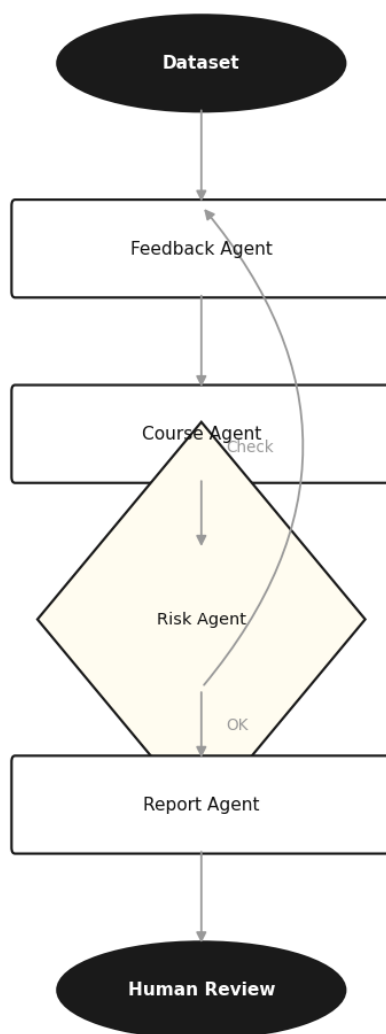


Figure 3: 智能体协作闭环：低置信或高风险结论需要回到上游重新检查，并进入人工复核。

4.2 编排原则

轻量分析任务应提供快速反馈，用于课程概览、数据质量提示和初步诊断。

复杂分析任务应采用异步处理机制，支持进度提示、失败重试、结果归档和审计记录。

高风险结论必须经过人工复核状态，不应直接进入正式评价报告。

每个智能体输出应包含证据标识、置信说明、风险提示和建议动作，便于后续追踪与复核。

五、数据与证据设计

5.1 碎片化信号串联

传统教育平台的数据往往是碎片化的：出勤记录在签到系统中，成绩在教务系统中，课堂互动在直播工具中，反馈在问卷系统中。EDUFISH 引擎的核心能力之一，是将这些碎片化信号实时捕获并串联为完整的教学质量证据链。

宿主平台通过 API 或 Webhook 将数据推送到引擎后，引擎自动完成字段归一化、时间对齐、关联匹配和证据索引构建。最终，一个学生的出勤变化、作业提交延迟和课堂互动频率下降，会被自动串联为一条可追溯的风险证据链，而不是三个孤立的数据点。

5.2 数据域

EDUFISH 将教育数据划分为稳定的数据域，以减少不同系统、不同表格、不同命名方式带来的解释偏差。首批数据域可覆盖课程、教师、学生、反馈、成绩和考勤，后续可扩展至作业、考试、课程资源和教学干预记录。

领域	典型字段	主要分析用途
courses	course_id, title, department, semester	课程画像、跨学期对比、质量趋势
teachers	teacher_id, name, department, role	教学表现、资源支持、课程责任人映射
students	student_id, program, cohort, group	群体差异、学习风险、匿名聚合
feedback	course_id, rating, comment, created_at	主题聚类、情绪识别、证据引用
grades	student_id, course_id, score, assessment	成绩分布、难度异常、学习成效
attendance	student_id, course_id, week, status	参与度、预警、教学节奏判断

表 6：首批教育数据域。

5.3 课程知识与证据库

课程知识库由课程材料、教学目标、评价标准、反馈证据和历史报告共同构成。不同课程具有不同的知识结构和评价重点，例如技术类课程更关注概念掌握、实践任务和项目质量，认知科学类课程更关注实验理解、理论框架和跨学科概念连接。

课程切换不只是更换资料库，还意味着评价目标、证据要求、分析模板和复核策略的同步切换。

JSON

```
{
  "course_id": "brain-cog-intro",
  "course_mode": "cognitive_science_intro",
  "rag_namespace": "courses/brain-cog-intro",
  "analysis_template": "course_quality_v1",
  "evidence_policy": {
    "require_citations": true,
    "min_sample_size": 12,
    "human_review_for_high_risk": true
  }
}
```

六、核心功能模块

EDUFISH 引擎的核心功能围绕教育数据进入引擎后的完整处理过程展开：先完成数据整理与质量校验，再形成可追溯证据，随后由智能体完成分析判断，最终生成面向不同角色的报告与改进建议。以下功能均为引擎级能力，宿主平台通过接入即可获得。

引擎能力	功能说明	宿主平台价值
数据模板管理	定义课程、反馈、成绩、考勤等数据域的字段规范和评价模板	保证不同课程和不同数据来源能够被一致理解
数据归一化	识别字段含义、统一命名、检查缺失值和异常分布	降低数据质量差异对分析结论的影响
智能分析预览	基于样本数据快速生成课程质量、反馈主题和风险提示	便于评审者快速把握课程运行状态
报告生成	根据受众角色生成摘要、证据、风险、建议和附录	提升正式评审材料的组织效率和专业一致性
证据追踪	为关键结论绑定指标、文本片段、时间范围和样本规模	支持复核、审计和后续改进追踪
干预策略推演	基于当前数据推演不同干预措施的预期效果	教师可直接看到增加实验复盘后达成率的提升预测

表 7：EDUFISH 引擎核心能力模块。

6.1 数据处理流程

处理阶段	说明
数据采集	接收宿主平台通过 API、SDK 或 Webhook 推送的课程材料、教务指标、学生反馈、成绩和考勤等多源数据
字段映射	将不同系统中的字段映射到统一教育数据模型
质量校验	检查缺失值、异常值、样本量不足和时间范围不一致等问题
证据绑定	将指标、文本片段和分析结论建立可追溯关联
结果生成	输出结构化洞察、风险提示、改进建议、干预策略推演和角色化报告

表 8：EDUFISH 数据处理流程。

6.2 分析任务结构

JSON

```
{
  "engine_scope": {
    "host_platform": "████████",
    "course_name": "██████",
    "semester": "██",
    "audience_role": "████ / █████ / ███"
  },
  "data_domains": ["██████", "██████", "██████", "██████"],
  "analysis_goals": ["██████", "██████", "██████", "██████"],
  "evidence_policy": {
    "require_traceable_sources": true,
    "show_confidence_level": true,
    "human_review_for_high_risk": true
  }
}
```

6.3 质量保障

为适应正式评审场景，EDUFISH 引擎的技术实现应把结果可靠性放在首位。质量保障不仅包括功能测试，还包括数据一致性、证据追踪、权限控制和异常结果复核。

机制	说明	作用
数据质量校验	识别字段缺失、样本不足、异常分布和时间范围不一致	避免低质量数据直接影响结论
证据链校验	检查关键结论是否绑定可追溯指标或文本材料	防止无依据的判断进入报告
角色权限控制	按评审者、教师、管理者等角色区分可见数据范围	保护学生与教师敏感信息
人工复核机制	高风险或低置信结论进入复核状态	保留人的判断权和最终解释权

表 9：EDUFISH 质量保障机制。

七、干预策略推演

7.1 推演机制

EDUFISH 引擎的差异化能力之一是干预策略推演。传统分析工具止步于“发现问题”，而 EDUFISH 能够基于当前数据和历史模式，推演不同干预措施的预期效果，让教师和管理者在执行干预前就能看到可能的改善路径。

推演过程由智能体协作完成：课程质量智能体提供当前状态基线，反馈分析智能体提供学生困惑聚类，风险治理智能体评估干预紧迫性，最终由推演模块生成多条干预路径及其预期效果曲线。

7.2 推演示例

以“脑与认知科学导论”课程为例：引擎检测到实验章节的学生达成率持续低于理论章节，同时反馈聚类显示“实验步骤不清晰”“缺少复盘环节”是主要困惑点。引擎自动推演以下干预策略及其预期效果：

干预策略	当前达成率	推演达成率	置信度
增加实验复盘环节	62%	78%	高
录制实验操作视频	62%	74%	中
调整实验分组方式	62%	69%	中
增加助教巡场频次	62%	71%	中

表 10：干预策略推演示例——基于当前数据分析的预期效果对比。

7.3 代码级实现

推演能力通过 EduPredictionService 实现，基于当前分析数据和历史模式进行动态预测。宿主平台可通过 API 直接调用推演接口，获取结构化的干预策略结果。

JSON

```
{
  "endpoint": "GET /api/edu/analysis/{id}/prediction",
  "response": {
    "scenarios": [
      {
        "name": "■■■■■■■",
        "current_rate": 0.62,
        "predicted_rate": 0.78,
        "confidence": "high",
        "evidence": ["feedback_cluster: experiment_review", "grade_trend: lab_chapters"]
      },
      {
        "name": "■■■■■■■",
        "current_rate": 0.62,
        "predicted_rate": 0.74,
        "confidence": "medium",
        "evidence": ["feedback_cluster: unclear_steps"]
      }
    ]
  }
}
```

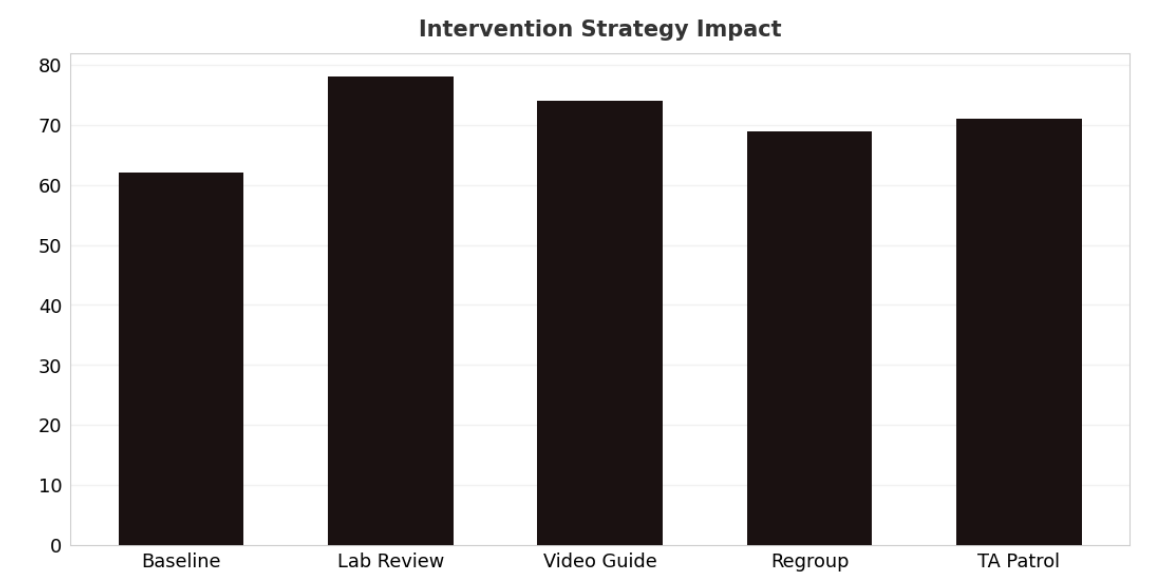


Figure 4: 干预策略推演效果对比：增加实验复盘环节的预期提升最为显著。

八、创新与可行性分析

EDUFISH 的创新点不在于简单生成报告，而在于以引擎形态将教育质量评估拆解为可治理的数据、可追溯的证据、可解释的智能分析、可推演的干预策略和可复核的改进建议。引擎架构使其具备极低的接入成本和极高的复用价值。

维度	说明	可行性判断
引擎架构	四层分离设计，API-First，宿主平台通过标准接口接入	已有 Flask API 后端和 Vue 前端，架构基础成熟
多智能体协同	不同分析任务由专用智能体分工处理	已实现 4 个专用智能体，SSE 流式通信
碎片化信号串联	实时捕获并关联出勤、成绩、课堂交互等多源数据	已有数据归一化和证据索引服务
干预策略推演	基于当前数据推演不同干预措施的预期效果	已有 EduPredictionService 和前端场景展示
证据图谱	自动生成可视化的教学质量证据图谱	已有 D3 + Canvas 双渲染器和 GSAP 动画系统
极低成本接入	SDK、Webhook、嵌入组件多种接入方式	后端 API 架构天然支持，Python/JS SDK 已交付，组件库已交付
报告生成	按角色生成摘要、风险、证据和建议	已有 EduReportService，支持 HTML 预览和 PDF 导出
企业级基础设施	API 版本化、认证、速率限制、Webhook、多租户、容器化	全部就绪，Phase 1-2 已交付

表 11：EDUFISH 创新点与可行性判断。

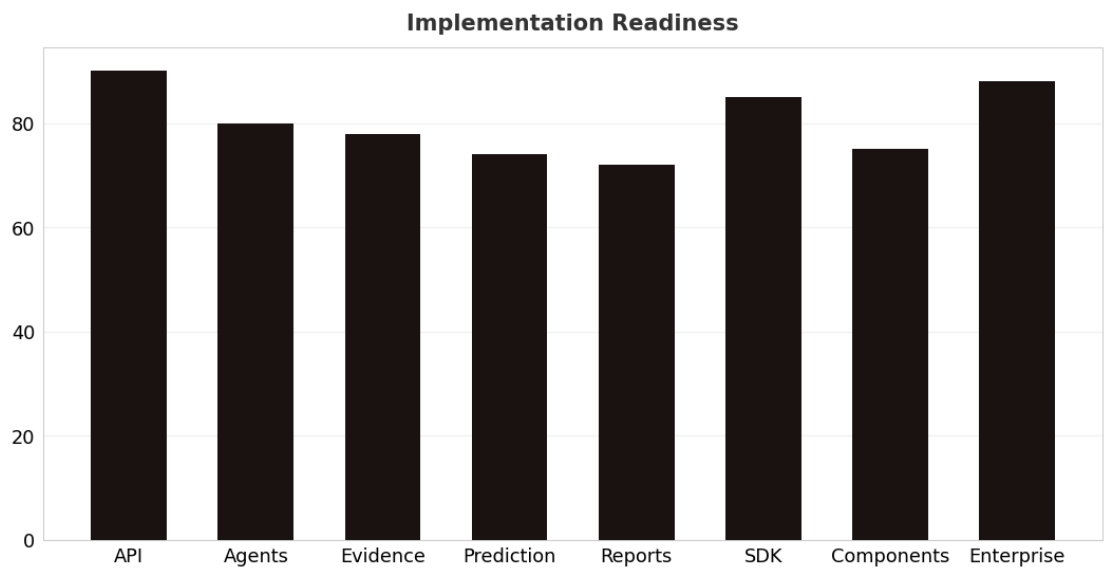


Figure 5: 引擎建设成熟度示意：核心分析能力和企业级基础设施已基本就绪，SDK、组件库、Webhook、多租户全部交付。

九、实施路线图

9.1 阶段目标

EDUFISH 引擎的建设按照“核心先行、接入跟进、多平台扩展”的路径推进。第一阶段完成引擎核心能力（数据治理与课程分析），第二阶段形成稳定的 API 接入层，第三阶段开发嵌入式前端组件，第四阶段适配多平台和扩展场景。

阶段	开发内容	验收标准
M1	引擎核心：数据治理与课程分析	已完成 — 数据归一化、字段映射、质量检查、证据绑定、课程质量诊断全部可用
M2	接入层：API 标准化与企业级特性	已完成 — /api/v1/ 版本化，API Key 认证，速率限制，Python/JS SDK，Webhook + HMAC 签名，多租户隔离，Docker 部署
M3	输出层：嵌入组件与报告	已完成 — 知识图谱组件库（ES/UMD），HTML 报告预览，PDF 导出，ResizeObserver 自适应，Web Worker 内联
M4	智能体协作深化	完成多智能体任务编排、置信说明、干预策略推演和高风险结论回溯
M5	多平台适配	支持 CSV/Excel 数据接入，适配主流教务系统数据格式，提供接入指南
M6	权限、隐私和审计	按角色访问，敏感数据脱敏，操作日志可追踪，引擎安全边界隔离

表 12：EDUFISH 引擎分阶段实施路线图。

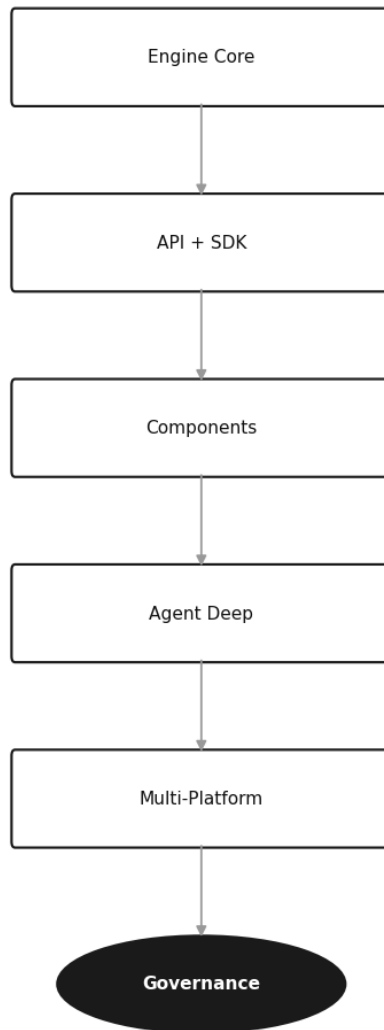


Figure 6: 实施顺序建议：先完成引擎核心和接入层，再开发嵌入组件和多平台适配。

9.2 应用场景与引擎能力

应用场景	引擎能力
宿主平台数据接入	API 数据推送、SDK 封装调用、Webhook 事件触发、字段自动映射
课程选择与分析模式切换	课程配置、评价模板、可用数据域和证据规则
动态分析概览	任务状态、智能体状态、证据节点和关系结构
干预策略推演	基于当前数据的多路径推演、预期效果曲线、置信度评估
报告预览和导出	结构化报告、引用证据、审计元数据和导出任务
复核与干预动作	结论状态、人工备注、任务分派和历史变更

表 13: EDUFISH 应用场景与引擎能力对应关系。

十、部署与运行说明

EDUFISH 引擎支持两种部署模式：独立部署模式下引擎作为独立服务运行，宿主平台通过 API 接入；嵌入部署模式下引擎核心服务与宿主平台共用基础设施，前端组件直接嵌入宿主平台界面。两种模式共享同一套引擎代码，区别仅在于网络拓扑和组件集成方式。

10.1 运行环境

组成	技术环境	说明
引擎后端	Python、Flask、SQLAlchemy	提供数据接入、智能分析、报告生成、干预推演和任务状态接口
前端应用	Vue 3、Vite、Pinia、D3、Three.js	负责评审工作台、课程视图、图谱展示和报告预览
数据存储	SQLite / PostgreSQL	本地演示可使用 SQLite，正式部署建议使用 PostgreSQL
证据与向量存储	ChromaDB 或同类向量数据库	用于课程材料、反馈文本和证据片段的检索与追踪
模型服务	OpenAI-compatible API	支持大语言模型与向量嵌入模型，可按机构部署策略切换服务商

表 14：EDUFISH 运行环境组成。

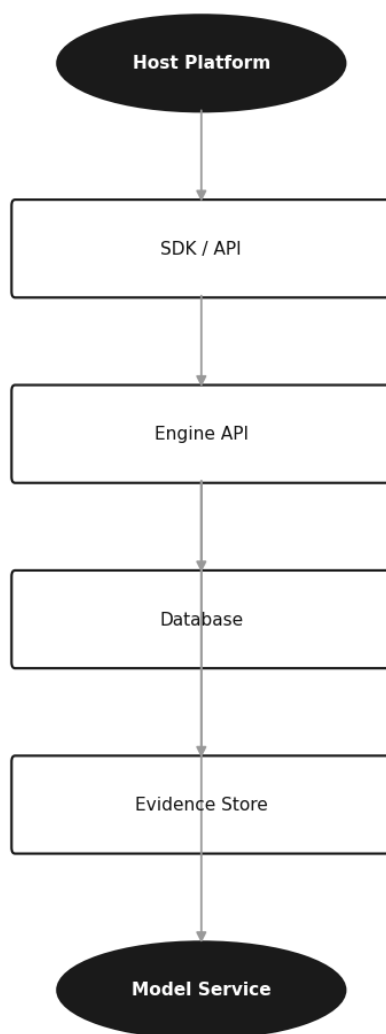


Figure 7: 引擎部署结构示意图：宿主平台通过 SDK/API 接入引擎，引擎后端连接数据库、证据存储和模型服务。

10.2 本地运行流程

本地运行主要用于评审演示、功能调试和开发验证。推荐使用统一命令完成依赖安装与服务启动，避免评审环境中出现前后端版本不一致的问题。

BASH

```
# █ █ ██████████ █
npm run setup      # ████████████████████
npm run dev        # ██████████3025████████5001█

# █ █ █████Docker █ █
cd deploy && docker compose up -d # ██████████
# █ █: http://localhost:3025
# █ █ API: http://localhost:5001/api/v1
# █ █ █ █: http://localhost:5001/health
```

开发模式下，前端服务运行在 3025 端口，引擎后端服务运行在 5001 端口；前端开发服务器会将 /api 与 /health 请求代理到后端（自动添加 /api/v1/ 前缀）。正式评审演示时，应提前完成数据准备和模型服务配置，避免现场等待材料解析或模型初始化。

10.3 构建与验证

BASH

```
# Run tests
npm run test

# Build production assets
npm run build

# Preview production build
cd frontend
npm run preview
```

构建验证应至少覆盖三类结果：引擎服务能正常启动，核心 API 接口能返回稳定结果，前端页面能完成课程选择、数据查看、分析预览、干预推演和报告阅读等主要流程。

10.4 生产部署建议

部署项	建议
引擎后端服务	使用 WSGI 服务运行 Flask 应用，并置于反向代理之后，独立于宿主平台部署。已提供 Dockerfile (python:3.11-slim) 和 docker-compose.yml 编排文件
前端静态资源	将构建后的 dist 目录交由 Nginx 或同类 Web 服务器托管，或通过 CDN 分发嵌入组件。组件库支持 npm 安装 (@edufish/vue)
数据库	已提供 SQLite 零配置启动，正式环境建议使用 PostgreSQL 独立实例，启用定期备份和权限隔离
文件与向量数据	上传材料、解析文本和向量索引应存储在可备份目录中，ChromaDB 支持独立向量存储

环境变量	通过 .env 文件或部署平台注入模型密钥、数据库连接和存储路径。提供 deploy/.env.template 模板文件
日志与监控	记录访问日志、错误日志、分析任务日志、Webhook 回调日志和人工复核操作日志。健康检查端点 /health 用于容器编排探活
一键部署	提供 deploy/setup.sh 脚本，自动完成 Python 依赖安装、前端构建、数据库初始化和服务启动

表 15：生产部署建议。

10.5 安全配置

配置项	说明
DATABASE_URL	数据库连接地址；正式环境不应使用默认本地文件数据库
UPLOAD_DIR	课程材料与上传文件目录；需设置访问权限和备份策略
LLM_API_KEY / LLM_BASE_URL	模型服务密钥与服务地址；密钥不得写入源码或报告
EMBEDDING_API_KEY / EMBEDDING_MODEL	向量嵌入模型配置；用于证据检索和材料索引
CHROMADB_DIR	向量数据库本地存储目录；需要与业务数据一起纳入备份计划
ENGINE_API_KEY	宿主平台接入引擎的认证密钥；需按宿主平台隔离权限

表 16：关键环境变量与安全说明。

部署原则：引擎独立部署时强调网络隔离和权限边界；嵌入部署时强调宿主平台数据主权和引擎安全边界。两种模式均需保护密钥、脱敏数据、审计日志和可备份恢复。

十一、人性化设计要求

11.1 教师不是数据录入员

教师工作界面要减少让用户填表的感觉。默认视图应该先呈现本周最重要的教学信号：学生困惑、课程节奏、风险预警、建议动作和相关证据。需要输入时，也应让输入附着在具体问题和上下文旁边，而不是放在孤立表单中。

11.2 管理者需要结论，但不能失去证据

院系管理者需要快速判断，但教育质量分析不能只给红黄绿灯。每个结论都应展开为三层：一句话判断、关键证据、建议动作。这样既能快速阅读，也能在争议场景中回到证据。

11.3 学生数据必须被克制使用

学生层面的分析应默认匿名聚合，只有在明确授权和必要场景下才进入个体干预。系统界面要区分风险提示、事实证据和推测性建议，避免把模型判断包装成确定结论。

人性化设计不是把页面做得柔和，而是减少认知负担、明确行动路径、保留人的判断权，并让系统的每一次建议都有出处。

十二、风险与治理

风险	表现	治理策略
数据偏差	小样本反馈被过度解释	展示样本量、置信度和数据缺口
隐私泄露	学生个体信息进入报告或模型提示	脱敏、角色权限、审计日志和最小化输入
模型幻觉	生成无证据结论或错误因果	强制 evidence_ids、引用检查和人工复核
过度自动化	管理者直接采纳 AI 建议	高风险结论进入待复核状态
视觉噪声	动画和网络图干扰判断	低频动效、悬停详情、默认静态可读
引擎安全边界	宿主平台数据意外暴露给其他实例	按宿主平台隔离数据存储、API 密钥和分析上下文
接入兼容性	不同宿主平台数据格式差异导致分析偏差	提供标准化数据映射模板和质量校验预警

表 17：EDUFISH 风险清单与治理措施。

12.1 审计字段

JSON

```
{
  "analysis_id": "edu_analysis_2026_0001",
  "host_platform": "platform_identifier",
  "data_version": "verified-dataset-version",
  "evidence_sources": ["student_feedback", "grade_distribution"],
  "confidence_level": "medium / high",
  "risk_flags": ["small_sample_size"],
  "review_status": "pending_review",
  "review_notes": "■■■■■■■■"
}
```

十三、结论

EDUFISH 智慧教育分析引擎面向传统教育平台数据分析滞后、干预手段单一、智能化改造成本高的痛点，提出了轻量化、可插拔的 AI 赋能方案。引擎通过多智能体协同、碎片化信号串联、证据图谱生成和干预策略推演，为宿主平台注入从数据到决策的完整智能闭环。

引擎建设已完成两个阶段：Phase 1 完成 API 标准化（认证、错误处理、校验、Python/JS SDK、Docker 容器化），Phase 2 完成企业级引擎特性（API 版本化、速率限制、Webhook 事件驱动、多租户隔离、前端组件库化）。当前引擎已具备生产环境多租户部署能力，核心分析链路（数据归一化 → 证据索引 → 智能体分析 → 报告生成 → 图谱可视化）全部打通，接入层（API/SDK/Webhook/组件）已稳定可用。后续方向为深化智能体协作、多平台数据格式适配和完善权限审计机制。

最终目标：让每一个传统教育系统都能以极低成本迅速拥有智能化的“大脑”。EDUFISH 不是另一个平台，而是一个引擎——接入即赋能，宿主平台瞬间获得从数据采集、证据建模、智能分析到干预推演的完整能力。

附录：术语说明

术语	说明
赋能引擎	不替代宿主系统，而是为其注入智能分析能力的可插拔技术组件
宿主平台	接入 EDUFISH 引擎的现有教育系统，保留全部用户关系、业务流程和数据主权
碎片化信号串联	将分散在不同系统中的出勤、成绩、课堂交互等数据关联为完整证据链的机制
干预策略推演	基于当前数据和历史模式，推演不同干预措施预期效果的分析能力
证据图谱	将指标、文本、材料来源和分析结论建立可追溯关系的可视化图谱
多智能体协作	将反馈分析、课程诊断、风险识别和报告生成拆分给不同智能体完成的分析方式
人工复核	对高风险、低置信或可能影响正式评价的结论进行人工确认
角色化报告	根据评审者、管理者和教师等不同阅读对象生成不同重点的报告

API 版本化	通过 /api/v1/ 前缀管理 API 版本，存量路径返回 410 引导迁移，响应头标注引擎版本和名称
速率限制	基于 Flask-Limiter 的 in-memory 请求频率保护，无需 Redis，可通过环境变量开关
Webhook 回调	异步任务完成后引擎主动向宿主平台发送签名的 HTTP POST 通知，含 HMAC-SHA256 校验
多租户隔离	通过 X-Tenant-ID 请求头实现数据按租户隔离，所有核心模型自动过滤，现有数据零丢失回填
组件库化	前端图谱组件通过 Vite library mode 构建为独立 npm 包，CSS 变量带 fallback，Worker 内联，支持 ES/UMD
容器化部署	引擎后端以 Docker 镜像分发，docker-compose.yml 编排一键启动，/health 用于容器探活

表 18: EDUFISH 文档术语说明。